



Access Control Policy

Lumi Education Pty Ltd · ABN 45 700 349 015 · 24 July 2026

DRAFT

ST4S items: A6 (access control), A7 (access review), A13 (least privilege / deny-by-default) **Related:** A5 (privileged-access MFA) **Version:** 0.1 DRAFT · **Date:** 2026-07-24 **Status:** Draft for review — not yet signed

1. Purpose and scope

This policy defines who may access what in Lumi Reading, how each role is authorized, and how access is kept least-privilege and reviewed. It covers the application's own access model — the Firestore/Storage security rules and server-owned role fields that govern parent, teacher, school-admin and super-admin access to child, class and school data — and the privileged operational access to the `lumi-ninc-au` project. Every control below is implemented and cited by file path.

Design principle: **authorization is server-defined, never client-asserted**. Every authorization decision derives from a server-written membership document or a server-owned field, not from a client-supplied claim or token attribute that the client could set itself. This was validated in the 2026-07 self-managed assessment (`docs/security/VULNERABILITY_ASSESSMENT_REPORT_2026-07-24.md`, §7 Positive assurance).

2. Role model

Role	Identity record	Scope of access
parent	<code>/schools/{schoolId}/parents/{uid}</code> (with <code>linkedChildren</code>)	Read/limited-write for their own linked children only, within one school
teacher	<code>/schools/{schoolId}/users/{uid}</code> with <code>role: 'teacher'</code>	Their own classes (via the class doc's <code>teacherId</code> / <code>teacherIds</code>) and those classes' students, within one school
schoolAdmin	<code>/schools/{schoolId}/users/{uid}</code> with <code>role: 'schoolAdmin'</code>	Their whole school — staff/parent directories, classes, students, allocations, settings
superAdmin	<code>/superAdmins/{uid}</code> (deny-all to clients)	Lumi internal operations across tenants, via the super-admin portal only

A signed-in principal with **no** membership document for a given school is denied — there is no ambient/default access.

3. How each role is authorized

3.1 Membership documents are server-written

Role comes from the membership document, and the security rules make those documents un-self-grantable (`firestore.rules`):

- **Teacher** (`/schools/{schoolId}/users/{userId}`): client teacher self-create was **removed**. A legitimate teacher document is written **server-side** (Admin SDK, bypassing rules) by the signup callables after a valid school-code check. The only client `create` paths that remain are an existing `schoolAdmin` provisioning staff, or the first-admin bootstrap of a school the caller just created (`firestore.rules:418-429`). This closes the historical teacher self-provisioning class — self-create previously let any account holder become a teacher of any school and read every child's PII.
- **Parent** (`/schools/{schoolId}/parents/{parentId}`): `allow create: if false` (`firestore.rules:480`). Parent profiles and their `linkedChildren` are created only server-side by the signup/linking callables after verifying a live child code — a bare signed-in account cannot join an arbitrary school id.
- **Self-updates are field-locked**: a user/parent may update only a small allowlist of own-profile fields (`preferences`, `fcmToken`, `lastLoginAt`, `relationshipLabel`, `characterId`, terms-acceptance). `role` and `schoolId` are pinned to their prior value on every self-update (`firestore.rules:436-460`, `487-508`). Entitlement/rate-limit/MFA-state fields are writable only from a trusted server source.

3.2 Role is resolved by reading the server doc, in-rules

The rules resolve a caller's role by reading their membership document (`getUserData` / `userRole`, `firestore.rules:16-42`) — `isSchoolAdmin`, `isTeacher`, `isParentMember` all derive from the server-written doc, never from a token claim. Teacher↔class ownership is resolved from the class document's `teacherId` / `teacherIds` (`teacherTeachesClass`, `firestore.rules:94-103`), the single source of truth for that relationship.

3.3 Super-admin is a Firestore doc, checked identically in three places

Super-admin status is the existence of a `/superAdmins/{uid}` document (with a `SUPER_ADMIN_UIDS` env value as a bootstrap-only fallback). The check is implemented three times, kept deliberately in sync:

- Portal session/auth — `admin/src/lib/auth-firestore.ts#isSuperAdminViaFirestore` (used by `admin/src/lib/auth.ts`, `admin-auth-guard.ts`).
- Cloud Functions — `functions/src/super_admin.ts#isSuperAdmin`.
- Privileged core — `packages/server-ops/src/authority.ts#assertSuperAdmin`.

The `/superAdmins/{uid}` collection is `allow read, write: if false` to all clients (`firestore.rules:1536-1537`), so it can only be managed server-side.

4. Deny-by-default posture (A13)

The rules are deny-by-default and defence-checked:

- **Implicit deny + explicit deny**. No path is readable/writable unless a rule grants it, and ~20+ server-only collections/subcollections carry an explicit `allow read, write: if false` (e.g. `/superAdmins`), and the server-owned docs at `firestore.rules:1140, 1264, 1284, 1368-1587`). There is no recursive wildcard grant.
- **Tenant isolation is structural**. Every per-school path binds `schoolId`, and there is **no** client `collectionGroup` rule, so an unscoped cross-tenant collection-group sweep cannot be authorised. This

was proven dynamically: a school-A teacher can neither read school-B data directly nor sweep it via a collection-group query (`functions/test/security_poc.rules.test.js` , the "S4 tenant isolation" test).

- **Server-owned fields are locked on create and update.** A shared server-owned-field denylist guards both paths, so an entitlement or ownership field cannot be forged at creation time (see §5).
- **Fail-closed entitlement.** `studentAccessLive` (`firestore.rules:134-138`) uses safe `.get(key, default)` accessors so a missing `access` map evaluates to **denied**, and an absolute `expiresAt` backstop lapses access on schedule even if no Cloud Function runs.

5. Deployed access-control hardening (F-01 ... F-04)

The 2026-07 assessment identified and **fixed + deployed** four access-control findings; each rules fix carries an emulator regression test wired into CI:

- **F-01 — student-create field-lock.** The student `create` rule omitted the server-owned-field denylist the `update` rule enforced, so a schoolAdmin/teacher could mint a student that is entitlement-live from creation, bypassing the single licensing-enforcement point. Fixed with one shared field list guarding create and update; regression test `security_poc.rules.test.js` (F-01 a/b/c). Deployed `firestore:rules` (PR #520 / `0d31809`).
- **F-02 — school-create commercial-field guard.** School `create` omitted the guard the `update` path enforced, letting a creator self-seed `subscription` / `access` / `accessMode` / `isDemo` . Fixed + regression test. Deployed.
- **F-03 — class ownership lock.** Class `update` let a teacher-of-class reassign `teacherId` / `teacherIds` (hand the class to an attacker uid or remove themselves). Fixed with a `classOwnershipUnchanged()` guard on the teacher branch (schoolAdmins retain reassignment); regression test. Deployed.
- **F-04 — in-module super-admin authz on destructive server-ops.** The five destructive super-admin operations (offboard a school, delete parent accounts, grant dev-access, disable staff auth, manage school-user auth) had no internal authorization — the only gate was the calling portal route's session check (single-layer). A fail-closed `assertSuperAdmin(db, actor.uid)` now runs **inside** the ops (`packages/server-ops/src/authority.ts`), so a route that ever forgets to gate fails **closed** instead of executing. Unit test `authority.parity.test.ts` . Deployed via `admin-deploy` (PR #561 / `6d5b795`).

Two lower-severity access-adjacent findings are **accepted-by-design** with written rationale: F-06 (Storage cover first-claim — Storage rules cannot read Firestore for role; covers are non-personal; uploader-uid and no-overwrite protections stand) and F-08 (portal CSRF resting on `SameSite=Lax`). See the findings register for dispositions.

6. Privileged access and MFA (A5)

- **Mandatory authenticator (TOTP) MFA for school administrators.** A school administrator must complete TOTP MFA before the portal issues its server session cookie; teachers retain password/SMS. **Source:** `docs/ADMIN_TOTP_MFA_RUNBOOK.md` .
 - A password-only or phone-MFA-only admin ID token is **rejected** by `/api/auth/session` until TOTP enrolment finishes; any admin cookie minted before the rollout is rejected and must be replaced by a new MFA-verified login. A valid TOTP login mints an HttpOnly, Secure (production), SameSite=Lax cookie.
 - Lost-device recovery is performed only by a separately authenticated support operator via the Admin SDK, recorded in the security audit log; an administrator can never disable their own final factor from an active session, and two active admin accounts are kept so recovery never depends on the locked-out person.

- Emergency rollback is the short-lived `ADMIN_TOTP_ENFORCED=false` flag only.
- Super-admin portal** authenticates against `/superAdmins/{uid}` (§3.3) with its own runtime service account and session secret; the seeded read-only sales-demo admin is the only MFA-exempt admin shape and is confined to a synthetic tenant, read-only (`firestore.rules:184-191`, `demoAdminReadOnly`).

7. Least privilege — infrastructure identities (A13)

- Dedicated runtime identities.** Cloud Functions run as `lumi-functions-runtime@lumi-ninc-au.iam.gserviceaccount.com` (not the default App Engine/appspot account); the two portals each have their own runtime service account. Scheduler/Eventarc-backed services grant `run.invoker` to the runtime identity per-service, and obsolete default-account bindings are removed (`docs/security/OPERATIONS_HEALTH_AUDIT_2026-07-17.md`).
- Keyless CI deploy.** The admin auto-deploy authenticates via GitHub OIDC / Workload Identity Federation — no JSON private key is stored; the provider condition accepts only this repo's `main` branch, and the SA binding is scoped to that principal. A predeploy identity audit (`infra/iam/audit-admin-build-identity.sh`) **refuses the deploy** if the build or runtime identity drifts to a default account or gains project-data/secret/key access (`.github/workflows/admin-deploy.yml`).
- No credentials in clients or builds.** The release gate forbids any secret, Admin credential or billable unrestricted key from entering a client, build artifact, log or Remote Config; secret scanning enforces it in CI (`.github/workflows/secret-scan.yml`; history is gitleaks-clean).

8. Access review cadence (A7)

Access type	Review trigger	Frequency	Evidence
Membership (parent/teacher/schoolAdmin)	On role change (provision / offboard) and on the school's annual rollover	Continuous + annual	Server-op offboarding; rollover import (soft-archive)
Super-admin (<code>/superAdmins</code>)	On any addition/removal; reviewed at least annually	Annual + on change	<code>/superAdmins</code> collection membership
IAM roles / service accounts / deploy WIF	Monthly and after any deployment change	Monthly	IAM export + capability canaries (release- gate schedule)
Privileged portal sessions / MFA factors	On staff change; lost- device recovery on demand	On change	<code>docs/ADMIN_TOTP_MFA_RUNBOOK.md</code>

Membership changes are **event-driven** (an admin provisions/offboards staff; a parent link is created/removed by callable; rollover soft-archives departed students), which keeps day-to-day access current between the calendar reviews. The **monthly IAM review** and the periodic privileged-access review are defined in the release-gate recurring-schedule table (`docs/privacy/RELEASE_PRIVACY_SECURITY_REVIEW.md`).

9. Known gaps (for the reviewer)

- Access register (A7).** This policy names *where* each role lives and *when* it is reviewed, but a standing **access register** (the enumerated list of current super-admins and privileged human accounts with review

dates) is a document Nic must own and keep — confirm it exists and is current.

- **Annual review sign-off.** The cadence table asserts annual super-admin and monthly IAM reviews; confirm these are actually being performed and recorded (not only documented), and that a named owner signs each off.
- **App Check (F-05).** Client attestation (App Check) enforcement on callables is currently **off** — a launch-gated control (valid store-signed attestation needs the published app). It is tracked separately; note it here so the policy is not read as claiming App Check is enforced today.
- **Sign-off.** This policy requires Nic's sign-off before it is treated as the governing document.